What is MongoDB?

- MongoDB is an open-source, cross-platform, and distributed document-based database designed for ease of application development and scaling. It is a NoSQL database developed by MongoDB Inc.
- MongoDB name is derived from the word "Humongous" which means huge, enormous. MongoDB database is built to store a huge amount of data and also perform fast.
- MongoDB is not a Relational Database Management System (RDBMS). It's called a "NoSQL" database. It is opposite to SQL based databases where it does not normalize data under schemas and tables where every table has a fixed structure. Instead, it stores data in the collections as JSON based documents and does not enforce schemas. It does not have tables, rows, and columns as other SQL (RDBMS) databases.
- The following table lists the relation between MongoDB and RDBMS terminologies.

MongoDB (NoSQL Database)	RDBMS (SQL Server, Oracle, etc.)
Database	Database
Collection	Table
Document	Row (Record)
Field	Column

- In the RDBMS database, a table can have multiple rows and columns. Similarly, in MongoDB, a collection can have multiple documents which are equivalent to the rows. Each document has multiple "fields" which are equivalent to the columns. Documents in a single collection can have different fields.

Advantages of MongoDB

1. **MongoDB stores data as JSON based document** that does not enforce the schema. It allows us to store hierarchical data in a document. This makes it easy to store and retrieve data in an efficient manner.
2. It is easy to scale up or down as per the requirement since it is a document based database. MongoDB also allows us to **split data across multiple servers.**
3. MongoDB provides rich features like **indexing, aggregation, file store**, etc.
4. MongoDB **performs fast with huge data**.
5. MongoDB **provides drivers to store and fetch data** from different applications developed in different technologies such as C#, Java, Python, Node.js, etc.
6. MongoDB **provides tools to manage MongoDB databases**.
7. Uses BSON type : BSON stands for Binary JSON and is a data format that MongoDB uses to store and organize data. BSON is a **binary serialization format** that includes all JSON data structure types and adds support for other types, such as dates, different size integers, ObjectIds, and binary data. BSON's binary-encoded serialization format encodes type and length information, which allows it to be traversed much more quickly compared to JSON.

MondoDB Shell Command:

- MongoDB Shell is the quickest way to connect, configure, query, and work with your MongoDB database. It acts as a command-line client of the MongoDB server.
- You can start MongoDB Shell by executing mongo or mongosh command on the command prompt/terminal. mongosh is the new MongoDB shell with some more features than the old mongo shell.

MongoDB Database:

- MongoDB is a document-oriented open-source NoSQL database. It is one of the most popular and widely used NoSQL databases.
- A database is a place where data is stored in an organized way. In MongoDB, databases are used to store collections. A single MongoDB server can have multiple databases and a single MongoDB database can have multiple collections.
- You can use MongoDB Shell or MongoDB Compass to create a new database.
- MongoDB provides the use <database-name command to connect with the database. If the specified database name does not exist, then it creates it and set it as a current database.
- For example, the following command switch to the "humanResouredb" database. If it does not exist, then creates it.

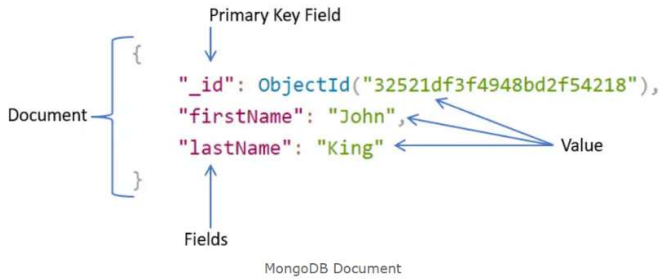
  use humanResourceDB

MongoDB Collections

- A collection in MongoDB is similar to a table in RDBMS. MongoDB collections do not enforce schemas. Each MongoDB collection can have multiple documents. A document is equivalent to row in a table in RDBMS.
- To create a collection, use the db.createCollection() command. The following creates a new employees collection in the current database
- Use the show collections commands to list all the collections in a database.
- To delete a collection, use the db.<collection-name>.drop() method.

MongoDB Documents: Document, Array, Embedded Document

- In the RDBMS database, a table can have multiple rows and columns. Similarly, in MongoDB, a collection can have multiple documents which are equivalent to the rows. Each document has multiple "fields" which are equivalent to the columns. So in simple terms, each MongoDB document is a record and a collection is a table that can store multiple documents.



- In the above example, a document is contained within the curly braces. It contains multiple fields in "field":"value" format. Above, "_id", "firstName", and "lastName" are field names with their respective values after a colon :. Fields are separated by a comma. A single collection can have multiple such documents separated by a comma.
- MongoDB stores data in key-value pairs as a BSON document. BSON is a binary representation of a JSON document that supports more data types than JSON. MongoDB drivers convert JSON document to BSON data.

MongoDB - Insert Single Document in a Collection using insertOne()

- In MongoDB, a collection represents a table in RDBMS and a document is like a record in a table. Learn how to insert a single document into a collection.
- MongoDB provides the following methods to insert documents into a collection:

1. insertOne() - Inserts a single document into a collection.
2. insert() - Inserts one or more documents into a collection.
3. insertMany() - Insert multiple documents into a collection.

insertOne()

- Use the db.<collection>.insertOne() method to insert a single document in a collection. db points to the current database, <collection> is existing or a new collection name.
- **Syntax:**

```
db.collection.insertOne(document)
```

The following inserts a document into employees collection.

- Example: insertOne()

```
db.employees.insertOne({
   firstName: "John",
   lastName: "King",
   email: "john.king@abc.com"
})
```

Output

```
{
  acknowledged: true,
  insertedId: ObjectId("616d44bea861820797edd9b0")
}
```

- In the above example, we passed a document to the insertOne() method. Notice that we haven't specified _id field. So, MongoDB inserts a document to a collection with the auto-generated unique _id field. It returns an object with a boolean

field acknowledged that indicates whether the insert operation is successful or not, and insertedId field with the newly inserted _id value.

- Use the find() to list all data of a collection, and the pretty() method to format resulted data.

```
db.employees.find().pretty()
```

Insert()

- The db.<collection>.insert() method inserts one document or an array of documents into a collection.
- **Syntax:**

```
db.collection.insert(
document or array of documents,[ordered])
```

- **Parameters:**

1. document or array of documents: A single document or array of documents to insert into the collection.
2. ordered: Optional. A boolean value indicating whether it's ordered or unordered insert operation. If true then performs ordered insert where if an error occurs with one of the documents, then MongoDB will return without processing the remaining documents in the array. If false, then process all the documents even if an error occurred. Defaults to true.

- **Example**

```
db.employees.insert(
    [
        {
            firstName: "John",
            lastName: "King",
            email: "john.king@abc.com"
        },
        {
            firstName: "Sachin",
            lastName: "T",
            email: "sachin.t@abc.com"
        },
        {
            firstName: "James",
            lastName: "Bond",
            email: "jamesb@abc.com"
        }
    ])
```

MongoDB: Find Single Document from Collection using findOne()

In MongoDB, a collection represents a table in RDBMS and a document is like a record in a table. Here you will learn how to retrieve or find a single document from a collection.

MongoDB provides two methods for finding documents from a collection:

1. findOne() - returns a the first document that matched with the specified criteria.
2. find() - returns a cursor to the selected documents that matched with the specified criteria.

findOne()

- The findOne() method returns the first document that is matched with the specified criteria.
- ***Syntax:***
  db.collection.findOne(query, projection)

- ***Parameters:***
  1. query: Optional. Specifies the criteria using query operators.
     - This parameter specifies the criteria used to select the document. It is a document that defines the conditions the retrieved document must meet.
     - Example: { userName: "john_doe" } will find the first document where userName is "john_doe".

  2. projection: Optional. Specifies the fields to include in a resulted document.

     - This parameter is used to specify or limit the fields that should be returned in the result.
     - It is a document where the keys represent field names and the values are either (to include the field) or 0 (to exclude the field).

 Example: { _id: 0, userName: 1, age: 1 } will include only the userName and age fields in the result, while excluding the _id field.

Example : db.users.findOne({ userName: "john_doe" }, { userName: 1, age: 1 });

The findOne() returns the first document from a collection if no parameter is passed.

MongoDB find() with $gt, $lt, $sort, and Other Options

The findOne() returns the first document from a collection if no parameter is passed.

MongoDB's find() method is used to query documents from a collection. It allows you to specify a set of conditions to filter the documents you want to retrieve. Additionally, MongoDB provides various **query operators** such as $gt, $lt, $eq, and **options** like $sort to fine-tune your queries.

Comparison Operators

1. **Using** $gt **(Greater Than)**
   Example: Retrieve all employees with a salary greater than 60,000.
   db.employees.find({ salary: { $gt: 60000 } });
2. Using $lt (Less Than)
   Example: Retrieve all employees younger than 30 years old.
   db.employees.find({ age: { $lt: 30 } });
3. Using $gte (Greater Than or Equal To)
   Example: Retrieve all employees with a salary greater than or equal to 50,000.
   db.employees.find({ salary: { $gte: 50000 } });
4. Using $lte (Less Than or Equal To)
   Example: Retrieve all employees older than or equal to 40 years old.
   db.employees.find({ age: { $lte: 40 } });
5. Using $eq (Equal To)
   Example: Retrieve all employees with a salary of exactly 75,000.
   db.employees.find({ salary: { $eq: 75000 } });
6. **Using $ne (Not Equal To)**
   Example: Retrieve all employees who do not work in the IT department.
   db.employees.find({ department: { $ne: "IT" } });
7. **Sorting with $sort**
   The $sort option is used to sort query results in either ascending or descending order. Sorting is based on the values of one or more fields.
   - 1: Sort in **ascending** order.
   - -1: Sort in **descending** order.
   - **Example 1**: Sort employees by salary in descending order.
     db.employees.find().sort({ salary: -1 });
   - **Example 2**: Sort employees by age in ascending order.
     db.employees.find().sort({ age: 1 });
   - **Example 3**: Sort employees first by department in ascending order, then by salary in descending order within each department
     db.employees.find().sort({ department: 1, salary: -1 });
8. **Limiting Results with .limit()**
   Example: Retrieve the first 3 employees with the highest salary.
   db.employees.find().sort({ salary: -1 }).limit(3);

Update Single Document using updateOne() in MongoDB

Update Command	Description	Example	Result
updateOne()	Updates a single document that matches the query.	db.users.updateOne({ userName: "john_doe" }, { $set: { age: 26 } })	Updates the age field of the first document where userName is "john_doe".
updateMany()	Updates all documents that match the query.	db.users.updateMany({ age: { $gt: 25 } }, { $set: { isActive: true } })	Sets isActive to true for all users older than 25.
replaceOne()	Replaces a document entirely, except for the _id field.	db.users.replaceOne({ userName: "john_doe" }, { userName: "john_doe", age: 27, city: "New York" })	Replaces the document where userName is "john_doe" with a new document (keeps the same _id).
findOneAndUpdate()	Finds a document and updates it, returning the original or updated document.	db.users.findOneAndUpdate({ userName: "john_doe" }, { $inc: { age: 1 } }, { returnOriginal: false })	Increments age by 1 for john_doe and returns the updated document.
$set	Updates the value of a field.	db.users.updateOne({ userName: "john_doe" }, { $set: { age: 26 } })	Sets the age field to 26.
$unset	Removes a field from the document.	db.users.updateOne({ userName: "john_doe" }, { $unset: { age: "" } })	Removes the age field from the document.
$inc	Increments (or decrements) a field by a specified value.	db.users.updateOne({ userName: "john_doe" }, { $inc: { age: 1 } })	Increments the age field by 1.
$push	Adds an element to an array field.	db.users.updateOne({ userName: "john_doe" }, { $push: { hobbies: "gaming" } })	Adds "gaming" to the hobbies array.
$pull	Removes an element from an array field.	db.users.updateOne({ userName: "john_doe" }, { $pull: { hobbies: "gaming" } })	Removes "gaming" from the hobbies array.
$addToSet	Adds an element to an array only if it doesn't already exist.	db.users.updateOne({ userName: "john_doe" }, { $addToSet: { hobbies: "reading" } })	Adds "reading" to the hobbies array only if it’s not already present.
$rename	Renames a field in the document.	db.users.updateOne({ userName: "john_doe" }, { $rename: { "userName": "username" } })	Renames the userName field to username.

$setOnInsert	Sets the value of a field if a new document is created by the upsert operation.	db.users.updateOne({ userName: "jane_doe" }, { $set: { age: 28 }, $setOnInsert: { createdAt: new Date() } }, { upsert: true })	If jane_doe doesn't exist, insert a new document with createdAt field.

MongoDB: Delete Documents in a Collection

MongoDB provides the following methods to delete one or more documents in a collection.

Command	Description	Example	Output
deleteOne(filter)	Deletes the first document that matches the specified filter.	db.collection.deleteOne({ name: "Alice" })	{ "acknowledged": true, "deletedCount": 1 }
deleteMany(filter)	Deletes all documents that match the specified filter.	db.collection.deleteMany({ age: { $gte: 30 } })	{ "acknowledged": true, "deletedCount": 3 }
deleteMany({})	Deletes all documents in the collection (use with caution).	db.collection.deleteMany({})	{ "acknowledged": true, "deletedCount": 10 }
remove(filter) (Deprecated)	Deletes documents that match the specified filter (replaced by deleteOne and deleteMany).	db.collection.remove({ age: { $lt: 20 } })	{ "nRemoved": 5 }

- db.collection.deleteOne() - Deletes the first matching document in a collection.
- db.collection.deleteMany() - Deletes all the matching documents in a collection.

db.collection.deleteOne()

Use the db.<collection>.deleteOne() method to delete the first documents that match with the specified filter criteria in a collection.

Syntax:
db.collection.deleteOne(filter, options)

Parameters:

1. filter: The selection criteria for the update, same as find() method.
2. options: Optional. May contains options for update behavior. It includes writeConcern, collation, and hint parameters.

In the above syntax, db points to the current database, <collection> points is an existing collection name.

```
db.employees.deleteOne({ salary:7000 })
```

deleteMany()

Use the db.<collection>.deleteMany() method to delete all the documents that match with the specified filter criteria in a collection.

Syntax:

db.collection.deleteMany(filter, options)

Parameters:

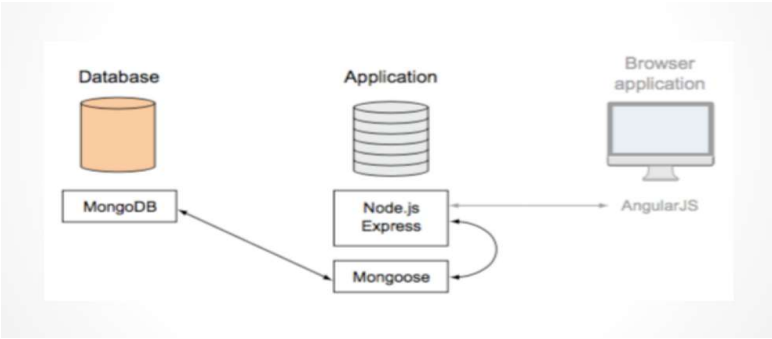
1. filter: The selection criteria for the update, same as find() method.
2. options: Optional. May contains options for update behavior. It includes writeConcern and collation, and hint parameters.

The following deletes all documents from the employees collection that match with the specified criteria.

```
db.employees.deleteMany({ salary:7000 })
```

Mongoose: A Powerful Tool for MongoDB

Mongoose is an Object Data Modeling (ODM) library for MongoDB, built on top of Node.js. It provides a robust and flexible way to interact with MongoDB databases, simplifying the process of creating, reading, updating, and deleting documents. This presentation will explore into the key features of Mongoose, exploring its capabilities for schema design, model creation, and advanced operations.



1. **Database (MongoDB)**

- **MongoDB** is a NoSQL database used to store the application's data. It holds collections and documents, where each document is a JSON-like structure.
- In this architecture, MongoDB is responsible for storing persistent data such as user information, product data, or any other relevant application data.

2. Application Layer

- The core of the application consists of **Node.js**, **ExpressJS**, and **Mongoose**.

Node.js:

- Node.js is a runtime environment that allows you to execute JavaScript on the server side. It provides the base for handling HTTP requests and managing connections.

ExpressJS:

- ExpressJS is a web framework built on top of Node.js, simplifying server-side development by providing built-in middleware, routing, and other utilities.
- It acts as a bridge between the **front-end browser application** (AngularJS) and the **back-end database** (MongoDB).

Mongoose:

- Mongoose is an ODM (Object Data Modeling) library for MongoDB, which helps to map the data from the MongoDB database into JavaScript objects.
- It is used to interact with MongoDB, handle database schemas, validations, and perform queries.

3. Browser Application (AngularJS)

- **AngularJS** is a front-end framework that provides the client-side logic for the web application. It handles the user interface, data binding, and communication with the server using AJAX.
- In this setup, AngularJS interacts with the ExpressJS API to fetch or post data.

Flow of Data in the System:

1. The browser (AngularJS) sends a request (e.g., fetch user data, submit a form) to the **ExpressJS** server running on Node.js.
2. **ExpressJS**, in turn, interacts with **Mongoose** to perform operations on the **MongoDB** database (e.g., fetch, insert, update, delete).
3. Mongoose communicates with MongoDB to execute the requested database operations.
4. The results from MongoDB are returned to **Mongoose**, then passed back to **ExpressJS**, which sends the data as a response to the **AngularJS** front-end.
5. Finally, the browser application (AngularJS) dynamically updates the UI based on the response received from the server.

Defining Your Data Structure: Mongoose Schemas

Mongoose schemas are the foundation for your MongoDB data structures. They define the shape and types of fields within your documents, ensuring data consistency and integrity. Schemas are created using the Mongoose `Schema` constructor, allowing you to specify each field's type, required status, default values, and more. For example, you can define a schema for a "Product" collection, specifying fields like `name`, `description`, `price`, and `category`.

Types: Mongoose supports various data types, including strings, numbers, booleans, dates, arrays, and embedded documents.

Validation: You can enforce data validation rules using schema validators, ensuring data quality and preventing invalid data from being saved.

Custom Types: Extend Mongoose's capabilities by creating custom data types for specific needs, such as a "phoneNumber" type with validation.

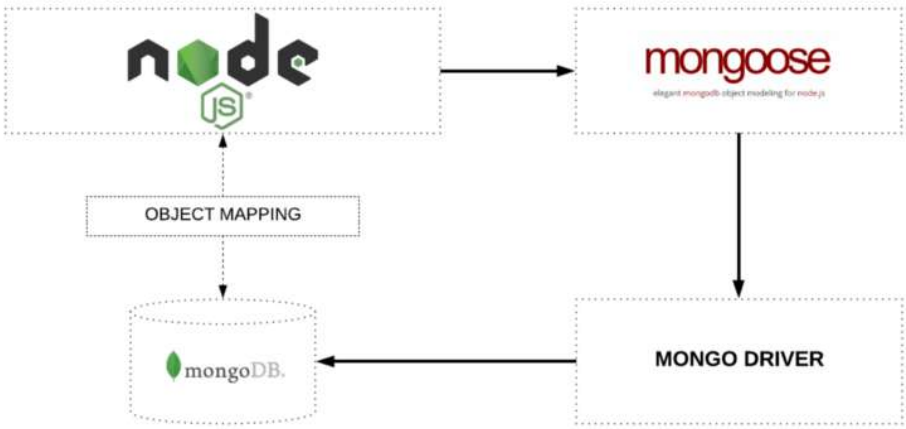
Bringing Schemas to Life: Mongoose Models

Mongoose models provide a convenient way to interact with your database collections. They are derived from schemas and offer methods for creating, retrieving, updating, and deleting documents. Models act as a bridge between your application code and the MongoDB database.

Creating Model: Use the `model()` function to create a model from a schema. The model name should correspond to the collection name in MongoDB.

Document Creation: Use the `create()` method to create new documents. The model provides an instance of the document, ready to be saved.

Retrieving Document: Use the `find()` method to query documents based on various criteria. You can filter, sort, and limit the returned results.



Collections

- *Collections* in Mongo are equivalent to tables in relational databases. They can hold multiple JSON documents.

Documents

- *Documents* are equivalent to records or rows of data in SQL. While a SQL row can reference data in other tables, Mongo documents usually combine that in a document.

Fields

- *Fields*, also known as properties or attributes, are similar to columns in a SQL table. In the image above, FirstName, LastName, Email, and Phone are all fields.

Schema

- While Mongo is schema-less, SQL defines a schema via the table definition. A Mongoose *schema* is a document data structure (or shape of the document) that is enforced via the application layer.

SchemaTypes

- While Mongoose schemas define the overall structure or shape of a document, *SchemaTypes* define the expected data type for individual fields (String, Number, Boolean, and so on).
- You can also pass in useful options like required to make a field non-optional, default to set a default value for the field, and many more.

Models

- *Models* are higher-order constructors that take a schema and create an instance of a document equivalent to records in a relational database.

Example: (app.js)

```
const puppySchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  age: Number
});

const Puppy = mongoose.model('Puppy', puppySchema);
```

- In the code above, puppySchema defines the shape of the document which has two fields, name, and age.
- The SchemaType for name is String and for age is Number. Note that you can define the SchemaType for a field by using an object with a type property like with name. Or you can apply a SchemaType directly to the field like with age.
- Also, notice that the SchemaType for name has the option required set to true. To use options like required and lowercase for a field, you need to use an object to set the SchemaType.
- At the bottom of the snippet, puppySchema is compiled into a model named Puppy, which can then be used to construct documents in an application.

Mongoose Connection String Options:

- Mongoose is a JavaScript library that provides a convenient way to interact with MongoDB databases. In order to use Mongoose, you need to create a connection between your Node.js application and the MongoDB server. The Mongoose connection represents this connection and provides a number of useful features, such as connection pooling and events for monitoring the connection status.
- Syntax
  mongoose.connect('<connection-string>', <options>, <callback>);

Steps to use a Mongoose model in a REST API with Express and Mongoose:

Step 1: Define a schema for the data model using Mongoose. The **schema** defines the structure of the data object, including its properties and data types.

- ✓ A schemaType is used to define the data type and validation rules for a specific field in a schema.

  Syntax:

```
const mySchema = new mongoose.Schema({
        myField: {
            type: MySchemaType,
            required: Boolean,
            default: Any,
            unique: Boolean,
            min: Number,
            max: Number,
            enum: Array,
            validate: Function
        }
});
```

Parameters:

- ✓ **Type:** This parameter is used to specify the data type of the field. For example, String, Number, Date, etc.
- ✓ **Required:** This parameter is used to specify whether the field is required or not. If set to true, Mongoose will throw a validation error if the field is not present in the document.
- ✓ **Default:** This parameter is used to set a default value for the field. If no value is provided for the field, Mongoose will use the default value.
- ✓ **Unique:** This parameter is used to specify whether the field value should be unique across all documents in the collection. If set to true, Mongoose will throw a validation error if a document with the same value for the field already exists.
- ✓ **Min/Max:** These parameters are used to set minimum and maximum values for numeric fields such as Number and Date. Mongoose will throw a validation error if the value of the field is outside the specified range.

- ✓ **Enum:** This parameter is used to specify a list of allowed values for the field. Mongoose will throw a validation error if the value of the field is not one of the allowed values.
- ✓ **Validate:** This parameter is used to specify a custom validation function for the field. The function should take the value of the field as its parameter and return true if the value is valid, or false otherwise. Mongoose will call this function during validation.

- Step 2: Create a Mongoose model using the schema. The model represents the data object in the database and provides methods for querying and updating it.

```
var userSchema = new mongoose.Schema({
   userID:String,
   userName:String,
   age:Number
})

var user = mongoose.model('user',userSchema,'userData')
```

- Use the Mongoose model in the route handlers to interact with the database. For example, we can use the model's find(), findOne(), create(), updateOne(), and deleteOne() methods to perform **CRUD (create, read, update, delete)** operations on the data object.

```
// Route to get all items
app.get('/api/items', (req, res) => {
   user.find().then((data)=>{
      res.json(data)
   })

});
//insert new item
app.post('/api/items',(req,res)=>{
   user.create({
      userID:req.body.userID,
      userName:req.body.userName,
      age:req.body.age
   }).then((newItem)=>{
      console.log(newItem)
      res.json(newItem)
   })
})
//delete item
app.delete('/api/delete/:id',(req,res)=>{
   let id = req.params.id
   user.deleteOne({userID:id}).then((err,data)=>{
      res.json(data)
         })
```

```
})
//update item
app.put('/api/update/:id',(req,res)=>{
    let id = req.params.id
    let updatedItemData = req.body
    user.updateOne({userID:id},updatedItemData).then((data)=>{
        res.json(data)
    })

})
```

Mongoose Queries:

Mongoose is a **MongoDB** object modelling and handling for node.js environment. **Mongoose Queries** are different static helper functions to carry out CRUD (Create Read Update and Delete) operations which are very important for any database. The static helper functions return a mongoose query object. The mongoose query is carried out first asynchronously and then calls the callback function passed to it. An alternative way is to use the synchronous function which is responsible for executing the query.

- **Model.deleteMany():** This query takes the parameters of any field that matches and then deletes all the entries in the database that matches.
- **Model.deleteOne():** This query takes the parameters of any field that matches and then deletes any one of the entries in the database that matches.
- **Model.find():** This query takes the parameters of one or more fields that match and then returns all the entries in the database that matches.
- **Model.findById()**: This query takes the id as the parameter and then returns the entry in the database if it exists matches.
- **Model.findByIdAndDelete()**: This query takes the id as the parameter and then deletes the entry in the database if it exists matches.
- **Model.findByIdAndRemove()**: This query takes the id as the parameter and then removes the entry in the database if it exists and then returns it to the callback function.
- **Model.findByIdAndUpdate()**: This query takes the id and the update parameters and values as the parameter and then updates the entry in the database if it exists.
- **Model.findOne():** This query takes the parameters of any field that matches and then returns any one of the entries in the database that matches.
- **Model.findOneAndDelete()**: This query takes the parameters of any field that matches and then returns and deletes any one of the entries in the database that matches.
- **Model.findOneAndRemove()**: This query takes the parameters of any field that matches and then returns and removes any one of the entries in the database that matches.
- **Model.findOneAndReplace()**: This query takes the parameters of any field and the replace document and then replaces any one of the entries in the database that matches.
- **Model.findOneAndUpdate()**: This query takes the parameters of one or more fields and the updated fields and values and then updates any one of the entries in the database that matches.
- **Model.replaceOne()**: This query takes the parameters as a filter and the replacement document and then replaces any one of the entries in the database that matches.

- **Model.updateMany()**: This query takes the parameters as a filter and the updating fields and values and then updates all of the entries in the database that matches.
- **Model.updateOne()**: This query takes the parameters as a filter and the updating fields and values and then updates any one of the entries in the database that matches.